

Taller de Git!

ComCom

DC - FCEyN - UBA

September 14, 2026



taller
de
git

Trabajando en grupos

IMPORTANTE

Hoy vamos a hacer varios ejercicios donde trabajamos de a grupos. Es importante que se sienten cerca de alguien con quien van a hacer unos ejercicios hoy. Vamos a configurar cosas de la compu y va a ser difícil mudarse mas tarde.

Recapitulando

La clase anterior, aprendimos cómo:

Recapitulando

La clase anterior, aprendimos cómo:

- Iniciar un repositorio vacío (`git...`).

Recapitulando

La clase anterior, aprendimos cómo:

- Iniciar un repositorio vacío (`git init`).

Recapitulando

La clase anterior, aprendimos cómo:

- Iniciar un repositorio vacío (`git init`).
- Marcar cambios como preparados o *staged* (`git...`), y confirmar estos cambios (`git...`).

Recapitulando

La clase anterior, aprendimos cómo:

- Iniciar un repositorio vacío (`git init`).
- Marcar cambios como preparados o *staged* (`git add`), y confirmar estos cambios (`git...`).

Recapitulando

La clase anterior, aprendimos cómo:

- Iniciar un repositorio vacío (`git init`).
- Marcar cambios como preparados o *staged* (`git add`), y confirmar estos cambios (`git commit`).

Recapitulando

La clase anterior, aprendimos cómo:

- Iniciar un repositorio vacío (`git init`).
- Marcar cambios como preparados o *staged* (`git add`), y confirmar estos cambios (`git commit`).
- Ver el estado actual de nuestros cambios (`git...`).

Recapitulando

La clase anterior, aprendimos cómo:

- Iniciar un repositorio vacío (`git init`).
- Marcar cambios como preparados o *staged* (`git add`), y confirmar estos cambios (`git commit`).
- Ver el estado actual de nuestros cambios (`git status`).

Otros comandos útiles

```
git rm
```

```
git mv
```

Otros comandos útiles

`git rm`

Permite borrar un archivo y marcar este cambio como *staged*. La sintaxis es `git rm [archivo]`.

`git mv`

Otros comandos útiles

git rm

Permite borrar un archivo y marcar este cambio como *staged*. La sintaxis es `git rm [archivo]`.

git mv

Permite mover/renombrar un archivo y marcar este cambio como *staged*. La sintaxis es `git mv [archivo] [nuevo nombre/ubicación]`.

Inspeccionando los cambios

```
git show
```

```
git diff
```

Inspeccionando los cambios

git show

Muestra el contenido del último commit. Esto incluye tanto el contenido (adiciones, borrados, etc) como metadatos (fecha, autor). Podemos pedirle que nos muestre el contenido de algún contenido específico puntual.

git diff

Inspeccionando los cambios

git show

Muestra el contenido del último commit. Esto incluye tanto el contenido (adiciones, borrados, etc) como metadatos (fecha, autor). Podemos pedirle que nos muestre el contenido de algún contenido específico puntual.

git diff

Muestra las diferencias entre el estado actual de los archivos y el último commit.

Si, en cambio, queremos ver las diferencias entre los cambios marcados como *staged* y los que confirmamos en el último *commit*, podemos usar `git diff --staged`.

Viendo la historia de los *commits*

git log

Viendo la historia de los *commits*

git log

Muestra el historial de *commits*.

Viendo la historia de los *commits*

git log

Muestra el historial de *commits*.

Ejercicio

Ejecutar `git log` en algún repo de la clase pasada.

Configuraciones iniciales 1/3

Tu identidad

Es importante establecer nuestro **nombre y email** en nuestro repositorio, ya que estos van a ir asociados con los cambios que hagamos:

```
git config --global user.name "Guybrush Threepwood"  
git config --global user.email guybrush@example.com
```

Modo de pull

Ejecuten esto (es una mauskiherramienta que nos ayudara mas tarde)

```
git config --global pull.rebase false
```

Config global vs local

También se puede omitir el flag `--global` para que las credenciales apliquen solo al repositorio en el que estamos trabajando.

Configuraciones iniciales 2/3

Clave SSH

Para poder trabajar cómodamente con repositorios Git que estén en Internet (GitHub, Bitbucket, GitLab, etc.), podemos configurar una **clave SSH** que nos identifique con el servidor que estemos usando. De esta manera, puede reconocer desde que usuario vienen los pedidos (para revisar permisos, etc...).

Configuraciones iniciales 3/3

Creando una clave nueva

Abrimos la terminal en el HOME y ejecutamos `cd .ssh` y luego `ssh-keygen -f id_rsa`. Le damos *Enter* a todo hasta que termine el proceso.

Configuraciones iniciales 3/3

Creando una clave nueva

Abrimos la terminal en el HOME y ejecutamos `cd .ssh` y luego `ssh-keygen -f id_rsa`. Le damos *Enter* a todo hasta que termine el proceso.

Subiendo la clave a COMCOMHub

- En la terminal, ejecutamos `cat id_rsa.pub` y copiamos todo lo que aparezca.
- Abrimos COMCOMHub, vamos a Configuración, Claves SSH / GPG, Añadir clave.
- Pegamos lo que tenemos copiado en el campo *Contenido*.

Configuraciones iniciales 3/3

Creando una clave nueva

Abrimos la terminal en el HOME y ejecutamos `cd .ssh` y luego `ssh-keygen -f id_rsa`. Le damos *Enter* a todo hasta que termine el proceso.

Subiendo la clave a COMCOMHub

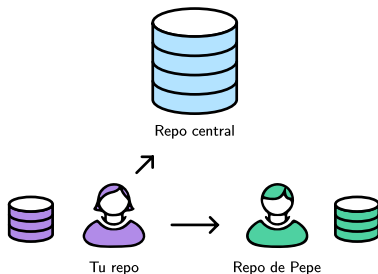
- En la terminal, ejecutamos `cat id_rsa.pub` y copiamos todo lo que aparezca.
- Abrimos COMCOMHub, vamos a Configuración, Claves SSH / GPG, Añadir clave.
- Pegamos lo que tenemos copiado en el campo *Contenido*.

Funcionó?

Pueden probar que la clave quedó funcionando usando el siguiente comando: `ssh -p 222 git@ssh.cubawiki.com.ar`. Deberían ver su username de COMCOMHub.

Colaborando con otras personas

Los repositorios remotos son *copias* de nuestro proyecto a las cuales accedemos a través de Internet. Puede haber varios, cada uno de los cuales puede ser de solo lectura o de lectura/escritura, según los permisos que tengamos.



Colaborar con otros implica gestionar estos repositorios remotos, y mandar (**push**) y recibir (**pull**) datos de ellos cuando necesites compartir cambios.

Usando Git online

Git server as a service

Para subir nuestros proyectos a Internet, ya sea para colaborar con otras personas o para compartir nuestro código, podemos usar un servicio de servidor web de Git.

Los más conocidos son Github, Bitbucket y Gitlab.



GitLab

También es posible hostear tu propia instancia de Git, por ejemplo git.cubawiki.com.ar.

Hay otros proyectos que comparten los *commits* por mail, por ejemplo.

Obteniendo un repositorio Git de otro lado

git clone

Para obtener una **copia local** de un repositorio existente en algún servidor, utilizamos el comando `git clone [URL]`. (hagan esto en el HOME, no en .ssh!)

En el caso de haber creado el repositorio con *git init*, esto no es necesario. El comando *clone* es como hacer *init + remote add + pull*.

Importante: *git clone [URL]* crea un nuevo directorio con el repositorio clonado. Debemos hacer *cd <repo>* luego de clonarlo!

¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas):



y



¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🌲 y 🎃

- 1 🌲: crear un repositorio nuevo en COMCOMHub, inicializando el README.md, y darle permiso a 🎃 para hacer *push* (por lo menos *Escritura*).

¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🌲 y 🎃

- 1 🌲: crear un repositorio nuevo en COMCOMHub, inicializando el README.md, y darle permiso a 🎃 para hacer *push* (por lo menos *Escritura*).
- 2 🌲 y 🎃: obtener una copia local del repositorio usando *git clone <URL SSH del repo>*.

¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🌲 y 🎃

- 1 🌲: crear un repositorio nuevo en COMCOMHub, inicializando el README.md, y darle permiso a 🎃 para hacer *push* (por lo menos *Escritura*).
- 2 🌲 y 🎃: obtener una copia local del repositorio usando *git clone <URL SSH del repo>*.
- 3 🌲: crear un archivo *navidad.txt* con unas palabras acerca de la navidad.
- 4 🎃: crear un archivo *halloween.txt* con unas palabras acerca de halloween.

¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🌲 y 🎃

- 1 🌲: crear un repositorio nuevo en COMCOMHub, inicializando el README.md, y darle permiso a 🎃 para hacer *push* (por lo menos *Escritura*).
- 2 🌲 y 🎃: obtener una copia local del repositorio usando *git clone <URL SSH del repo>*.
- 3 🌲: crear un archivo *navidad.txt* con unas palabras acerca de la navidad.
- 4 🎃: crear un archivo *halloween.txt* con unas palabras acerca de halloween.
- 5 🌲 y 🎃: hagan *commits* con sus cambios respectivos.

¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🌲 y 🎃

- 1 🌲: crear un repositorio nuevo en COMCOMHub, inicializando el README.md, y darle permiso a 🎃 para hacer *push* (por lo menos *Escritura*).
- 2 🌲 y 🎃: obtener una copia local del repositorio usando *git clone <URL SSH del repo>*.
- 3 🌲: crear un archivo *navidad.txt* con unas palabras acerca de la navidad.
- 4 🎃: crear un archivo *halloween.txt* con unas palabras acerca de halloween.
- 5 🌲 y 🎃: hagan *commits* con sus cambios respectivos.
- 6 🌲: hacer *git push* con sus cambios.

¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🌲 y 🎃

- 1 🌲: crear un repositorio nuevo en COMCOMHub, inicializando el README.md, y darle permiso a 🎃 para hacer *push* (por lo menos *Escritura*).
- 2 🌲 y 🎃: obtener una copia local del repositorio usando *git clone <URL SSH del repo>*.
- 3 🌲: crear un archivo *navidad.txt* con unas palabras acerca de la navidad.
- 4 🎃: crear un archivo *halloween.txt* con unas palabras acerca de halloween.
- 5 🌲 y 🎃: hagan *commits* con sus cambios respectivos.
- 6 🌲: hacer *git push* con sus cambios.
- 7 🎃: intentar hacer *git push* de los cambios al repositorio remoto. ¿Qué pasó?

¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🌲 y 🎃

- 1 🌲: crear un repositorio nuevo en COMCOMHub, inicializando el README.md, y darle permiso a 🎃 para hacer *push* (por lo menos *Escritura*).
- 2 🌲 y 🎃: obtener una copia local del repositorio usando *git clone <URL SSH del repo>*.
- 3 🌲: crear un archivo *navidad.txt* con unas palabras acerca de la navidad.
- 4 🎃: crear un archivo *halloween.txt* con unas palabras acerca de halloween.
- 5 🌲 y 🎃: hagan *commits* con sus cambios respectivos.
- 6 🌲: hacer *git push* con sus cambios.
- 7 🎃: intentar hacer *git push* de los cambios al repositorio remoto. ¿Qué pasó?
- 8 🎃: bajarse los cambios del repositorio remoto. con *git pull*. ¿Anduvo?

¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🌲 y 🎃

- 1 🌲: crear un repositorio nuevo en COMCOMHub, inicializando el README.md, y darle permiso a 🎃 para hacer *push* (por lo menos *Escritura*).
- 2 🌲 y 🎃: obtener una copia local del repositorio usando *git clone <URL SSH del repo>*.
- 3 🌲: crear un archivo *navidad.txt* con unas palabras acerca de la navidad.
- 4 🎃: crear un archivo *halloween.txt* con unas palabras acerca de halloween.
- 5 🌲 y 🎃: hagan *commits* con sus cambios respectivos.
- 6 🌲: hacer *git push* con sus cambios.
- 7 🎃: intentar hacer *git push* de los cambios al repositorio remoto. ¿Qué pasó?
- 8 🎃: bajarse los cambios del repositorio remoto. con *git pull*. ¿Anduvo?
- 9 🎃: *pushear* el resultado.

¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🌲 y 🎃

- 1 🌲: crear un repositorio nuevo en COMCOMHub, inicializando el README.md, y darle permiso a 🎃 para hacer *push* (por lo menos *Escritura*).
- 2 🌲 y 🎃: obtener una copia local del repositorio usando *git clone <URL SSH del repo>*.
- 3 🌲: crear un archivo *navidad.txt* con unas palabras acerca de la navidad.
- 4 🎃: crear un archivo *halloween.txt* con unas palabras acerca de halloween.
- 5 🌲 y 🎃: hagan *commits* con sus cambios respectivos.
- 6 🌲: hacer *git push* con sus cambios.
- 7 🎃: intentar hacer *git push* de los cambios al repositorio remoto. ¿Qué pasó?
- 8 🎃: bajarse los cambios del repositorio remoto. con *git pull*. ¿Anduvo?
- 9 🎃: *pushear* el resultado.
- 10 🌲: bajarse los nuevos cambios.

Visualizando los cambios 1/2

1) Commits independientes

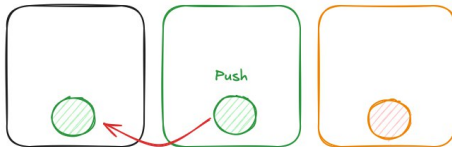


Visualizando los cambios 1/2

1) Commits independientes



2) Primer push

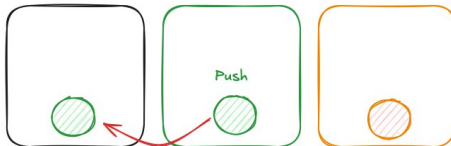


Visualizando los cambios 1/2

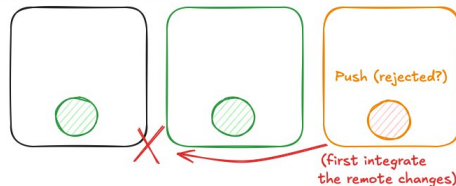
1) Commits independientes



2) Primer push

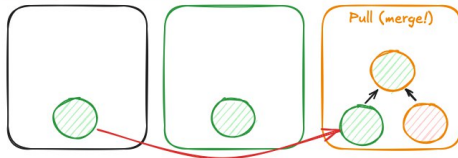


3) Segundo push rejected



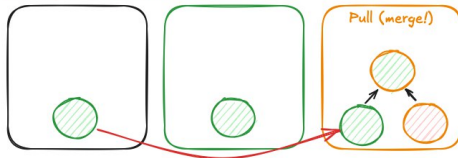
Visualizando los cambios 2/2

4) Pull para traer los cambios del remote

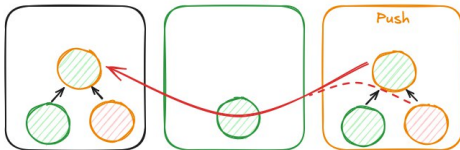


Visualizando los cambios 2/2

4) Pull para traer los cambios del remote

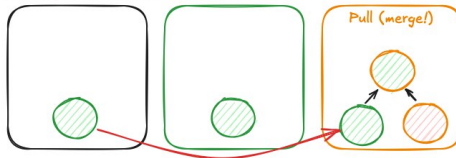


5) Push ahora funciona (no estamos pisando nada)

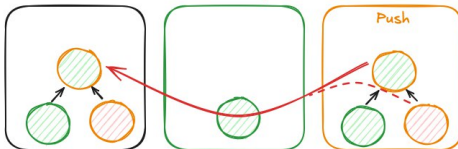


Visualizando los cambios 2/2

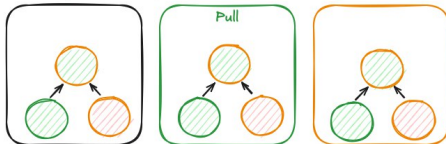
4) Pull para traer los cambios del remote



5) Push ahora funciona (no estamos pisando nada)



6) Pull para quedar iguales



Extra: vinculando un repositorio remoto

git remote

Extra: vinculando un repositorio remoto

git remote

Ver los repositorios remotos asociados

Ejecutamos `git remote -v`. Si no hay ningún output, es que el repo no tiene ningún remote. Es decir, no apunta a ningún otro repo fuera del local.

Extra: vinculando un repositorio remoto

git remote

Ver los repositorios remotos asociados

Ejecutamos `git remote -v`. Si no hay ningún output, es que el repo no tiene ningún remote. Es decir, no apunta a ningún otro repo fuera del local.

Agregar un repositorio remoto

Ejecutamos `git remote add origin [URL]`.

Extra: vinculando un repositorio remoto

git remote

Ver los repositorios remotos asociados

Ejecutamos `git remote -v`. Si no hay ningún output, es que el repo no tiene ningún remote. Es decir, no apunta a ningún otro repo fuera del local.

Agregar un repositorio remoto

Ejecutamos `git remote add origin [URL]`.

git clone ya lo hizo

Si tenemos un repo que iniciamos con `git clone`, ya tenemos configurado el remote llamado `origin` automáticamente.

Mandando fruta

Ejercicio de a 2 personas en máquinas: 🍇 y 🍐

Mandando fruta

Ejercicio de a 2 personas en máquinas: 🍇 y 🍐

- 1 🍇 y 🍐 : vamos a seguir usando el repo anterior (el de 🎄 y 🎃), asegurense de pullear lo ultimo asi seguimos desde ahí.

Mandando fruta

Ejercicio de a 2 personas en máquinas: 🍇 y 🍐

- 1 🍇 y 🍐 : vamos a seguir usando el repo anterior (el de 🌲 y 🎃), asegurense de pullear lo ultimo asi seguimos desde ahí.
- 2 🍇 : crear un archivo *ensalada.txt* con una linea que diga "bowl". aca va a ir la ensalada. commitear y pushear.
- 3 🍐 : pullear para obtener el archivo *ensalada.txt* con solo "bowl".

Mandando fruta

Ejercicio de a 2 personas en máquinas: 🍇 y 🍐

- 1 🍇 y 🍐 : vamos a seguir usando el repo anterior (el de 🌲 y 🎃), asegurense de pullear lo ultimo asi seguimos desde ahí.
- 2 🍇 : crear un archivo *ensalada.txt* con una linea que diga "bowl". aca va a ir la ensalada. commitear y pushear.
- 3 🍐 : pullear para obtener el archivo *ensalada.txt* con solo "bowl".
- 4 🍇 y 🍐 : hagan *commits* (1 cada) agregando algunos nombres de frutas a su ensalada en la misma linea que "bowl".

Mandando fruta

Ejercicio de a 2 personas en máquinas: 🍇 y 🍐

- 1 🍇 y 🍐 : vamos a seguir usando el repo anterior (el de 🌲 y 🎃), asegurense de pullear lo ultimo asi seguimos desde ahí.
- 2 🍇 : crear un archivo *ensalada.txt* con una linea que diga "bowl". aca va a ir la ensalada. commitear y pushear.
- 3 🍐 : pullear para obtener el archivo *ensalada.txt* con solo "bowl".
- 4 🍇 y 🍐 : hagan *commits* (1 cada) agregando algunos nombres de frutas a su ensalada en la misma linea que "bowl".
- 5 🍇 : hacer *git push* con sus cambios.

Mandando fruta

Ejercicio de a 2 personas en máquinas: 🍇 y 🍐

- 1 🍇 y 🍐 : vamos a seguir usando el repo anterior (el de 🌲 y 🎃), asegurense de pullear lo ultimo asi seguimos desde ahí.
- 2 🍇 : crear un archivo *ensalada.txt* con una linea que diga "bowl". aca va a ir la ensalada. commitear y pushear.
- 3 🍐 : pullear para obtener el archivo *ensalada.txt* con solo "bowl".
- 4 🍇 y 🍐 : hagan *commits* (1 cada) agregando algunos nombres de frutas a su ensalada en la misma linea que "bowl".
- 5 🍇 : hacer *git push* con sus cambios.
- 6 🍐 : bajarse los cambios del repositorio remoto con *git pull*. Resolver el conflicto (!) y pushear

Mandando fruta

Ejercicio de a 2 personas en máquinas: 🍇 y 🍐

- 1 🍇 y 🍐 : vamos a seguir usando el repo anterior (el de 🌲 y 🎃), asegurense de pullear lo ultimo asi seguimos desde ahí.
- 2 🍇 : crear un archivo *ensalada.txt* con una linea que diga "bowl". aca va a ir la ensalada. commitear y pushear.
- 3 🍐 : pullear para obtener el archivo *ensalada.txt* con solo "bowl".
- 4 🍇 y 🍐 : hagan *commits* (1 cada) agregando algunos nombres de frutas a su ensalada en la misma linea que "bowl".
- 5 🍇 : hacer *git push* con sus cambios.
- 6 🍐 : bajarse los cambios del repositorio remoto con *git pull*. Resolver el conflicto (!) y pushear
- 7 🍇 : bajarse los nuevos cambios.

Mandando fruta

Ejercicio de a 2 personas en máquinas: 🍇 y 🍐

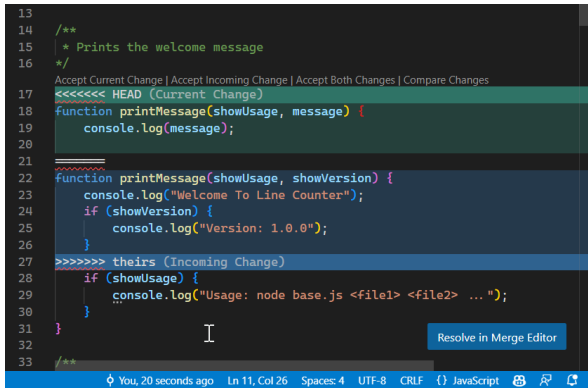
- 1 🍇 y 🍐 : vamos a seguir usando el repo anterior (el de 🌲 y 🎃), asegurense de pullear lo ultimo asi seguimos desde ahí.
- 2 🍇 : crear un archivo *ensalada.txt* con una linea que diga "bowl". aca va a ir la ensalada. commitear y pushear.
- 3 🍐 : pullear para obtener el archivo *ensalada.txt* con solo "bowl".
- 4 🍇 y 🍐 : hagan *commits* (1 cada) agregando algunos nombres de frutas a su ensalada en la misma linea que "bowl".
- 5 🍇 : hacer *git push* con sus cambios.
- 6 🍐 : bajarse los cambios del repositorio remoto con *git pull*. Resolver el conflicto (!) y pushear
- 7 🍇 : bajarse los nuevos cambios.
- 8 🍇 y 🍐 : repetir estos pasos con los roles invertidos si aun no lo hicieron (en un nuevo archivo).

Ejemplo de resolución de conflicto

```
$ git pull
Auto-merging ensalada.txt
CONFLICT (add/add): Merge conflict in ensalada.txt
Automatic merge failed; fix conflicts and then commit the result
$ cat ensalada.txt
<<<<<< HEAD
bowl pera
=====
bowl manzana
>>>>>> a473efc136d4fadf4d29476088b64dcbc82e8c03
$ nano ensalada.txt
$ cat ensalada.txt
bowl pera manzana
$ git add ensalada.txt
$ git commit -m "unificar ensaladas"
[main 64a1772] unificar ensaladas
$ git push
```

Comentario sobre otros editores...

Esto de corregir los conflictos "a mano" viene de estar utilizando `nano` como editor. Otros editores de código tienen distintas *features*, una de las cuales puede ser interpretar el formato de Git para mostrar los conflictos. Por ejemplo, Visual Studio Code muestra los conflictos con una interfaz gráfica y *botoncitos*.

A screenshot of the Visual Studio Code editor interface showing a merge conflict. The editor has a dark theme. At the top, there's a menu bar with options: 'Accept Current Change', 'Accept Incoming Change', 'Accept Both Changes', and 'Compare Changes'. Below this, the conflict is displayed with two versions of a file. The 'HEAD (Current Change)' version is highlighted in green and shows a function `printMessage(showUsage, message)` that logs the message. The 'theirs (Incoming Change)' version is highlighted in blue and shows a function `printMessage(showUsage, showVersion)` that logs a welcome message and the version. A cursor is positioned at the end of the 'theirs' version. At the bottom right, there is a button labeled 'Resolve in Merge Editor'. The status bar at the bottom shows 'You, 20 seconds ago', 'Ln 11, Col 26', 'Spaces: 4', 'UTF-8', 'CRLF', and the file type 'JavaScript'.

Muy importante (?)

